

README

Part 16: 图像生成 与 视频生成 — 扩散模型与跨模态对齐

- > 理解侧（Part 15）会"看"之后，本部分走生成侧：**文生图、图生图、参考图条件、
- > 文生视频——并沿一条主线贯穿：跨模态特征对齐**（文本/参考图特征如何"指挥"
- > 扩散网络生成）。锚点工具：[\[huggingface/diffusers\]](https://github.com/huggingface/diffusers)(<https://github.com/huggingface/diffusers>) (34.4k)
- > · 量化/小模型路径见各章。

学习目标

完成本部分后，你将能够：

- 理解 图像/视频生成在 LLM 链路中的位置和价值
- 手写 DDPM 的完整数学（前向闭式 + ϵ 预测训练 + 采样循环）并解释其工程权衡
- 配置 diffusers 的推理服务并理解每个参数的含义
- 完成 文生图、图生图、文生视频任务
- 识别 图像/视频生成中的常见陷阱并设计防范策略

理论背景（导览）

为什么需要生成侧？多模态理解（Part 15）只能"看图说话"，不能"按话画图"——生成侧补上"创造"的能力：扩散模型把"从噪声逐步去噪出数据"变成可训练、可控的目标。

DDPM 三段数学速览（记号约定： $\alpha_t = 1 - \beta_t$ 为单步信号保留率； $\bar{\alpha}_t = \prod_i \alpha_i$ 为累积保留率）：

机器	公式	一句话
前向加噪	$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon$	边际闭式一步到位——训练高效的全部秘密
反向去噪	$x_{t-1} = 1/\sqrt{\alpha_t} \cdot (x_t - \beta_t/\sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon^{\wedge}) + \sqrt{\beta_t} \cdot z$	t=0 时 z=0（最后一步不加噪）
条件注入	文本/参考图嵌入 → cross-attention 的 K/V	一个模态的表征"指挥"另一个模态生成

- > 完整的问题引入（vs GAN/VAE）、逐步推导、直觉解释与 2D 玩具验证，
- > 见 [01 章 · 手写 DDPM](01_ddpm_from_scratch.md)——本 README 不重复展开。

章节导航

章节	内容	对应脚本
[手写 DDPM](01_ddpm_from_scratch.md)	前向闭式 / ϵ 预测训练 / 采样循环——2D 玩具分布上全流程	'01'
[文生图与图生图](02_t2i_i2i_pipelines.md)	Latent Diffusion → SD/SDXL/SD3/FLUX 工具链 → img2img strength → ControlNet	—（diffusers 实
[特征对齐与视频生成](03_alignment_and_video.md)	IP-Adapter 解耦 KV / CFG / InstantID、PuLID → CogVideoX-2B 与 Wan2.1-1.3B	'02'

前置知识

- 必须掌握：
- 高斯分布的加法（两个高斯之和仍是高斯，方差相加）——01 章前向闭式的全部数学基础
- 建议掌握：
- Part 6：cross-attention（02/03 章生成条件注入的地基）；Part 8 06 章：VAE/量化概念（02 章 SD 的 fp16 显存策略与它同源）
- 可选：
- 无扩散基础也可直接开始：01 章 from scratch，三段数学全部现推

在 LLM 链路中的位置

Part 15（理解侧：图→文 对齐） $\rightleftharpoons$ 【本部分: 生成侧（文/图 → 图/视频）】

共通主线：跨模态特征对齐——把一个模态的表征"翻译"成另一个模态能消费的 token

环境与版本策略	
环境	说明
手写)	**CPU 可跑、零新依赖** 数学机制，玩具规模
users)	独立 venv：`pip install diffusers transformers accelerate` (latest) SD1.5 fp16 ~2GB (△ 用镜像 `stable-diffusion-v1-5/stable-diffusion-v1-5`, runwayml/stable-diffusion-v1-5) CogVideoX-2B (fp16 ~4GB) / Wan2.1-1.3B (8.2GB) 24GB 全兼容；量化路径更低

学习地图

手写 DDPM（01：前向闭式→训练→采样，2D 玩具）← 点（扩散数学）
↓ "搬到 VAE 潜空间 + cross-attention 条件"
文生图工具链（02：SD→SDXL→FLUX）+ img2img ← 线
↓ 参考条件注入
解耦交叉注意力（03：对齐机制手写）/ ControlNet（02 § 4）← 面
↓ 图像潜变量 → 时空潜变量
视频生成（03：CogVideoX / Wan2.1）→ 面试/工作就绪

课后作业

[Assignment 16](../assignments/assignment_16/)

相关资源

- [diffusers](https://github.com/huggingface/diffusers) (docs/ 概念指南是扩散最好教程)
- [ControlNet](https://github.com/lllyasviel/ControlNet) (34.1k) ·
- [IP-Adapter](https://github.com/tencent-ailab/IP-Adapter) ·

[InstantID](https://github.com/instantX-research/InstantID)
• [CogVideo](https://github.com/zai-org/CogVideo) · [Wan2.1](https://github.com/Wan-Video/Wan2.1) · [HunyuanVideo](https://github.com/Tencent-Hunyuan/HunyuanVideo)
• DDPM (2006.11239) · LDM (2112.10752) · SD3/Rectified Flow (2403.03206) · IP-Adapter (2308.06721) · CogVideoX (2408.06072) · Wan2.1 (2503.20314)

[← 上一章：Part 15 VLM](../Part15_vision_language/tutorial/README.md) |
[返回课程总览](../README.md) | [下一站：Part 17 Agentic RL
→](../Part17_agentic_rl/tutorial/README.md)

01_ddpm_from_scratch

01 — 手写 DDPM：扩散模型的三段数学

> 文生图的"引擎"是扩散模型。本章在 2D 玩具分布上手写它的完整数学
> (跑 [scripts/01_ddpm_from_scratch.py](../scripts/01_ddpm_from_scratch.py), CPU 30 秒) :
> 前向闭式加噪 $\rightarrow \epsilon$ 预测训练 $\rightarrow$ 反向采样循环。机制与 512×512 图像生成完全同构,
> 只是维度小到能看清每一步。

学习目标

完成本章后，你将能够：

- 手写 DDPM 的完整数学（前向闭式 + ϵ 预测训练 + 采样循环）
- 解释 前向闭式的物理含义（信号保留比例）
- 画出 扩散模型的数据流并标注每步 shape
- 识别 β schedule 选择、方差口径等常见陷阱

前置知识

必须掌握：

- 高斯分布的加法（两个高斯之和仍是高斯，方差相加）——前向闭式的全部数学基础，本章数学全部现推，无扩散基础也能跟上

建议掌握：

- [Part 6 02 章 · Attention 从零开始](../Part6_transformer/tutorial/02_attention_from_scratch.md)——
Q · K/V"按相似度挑选信息"的机制是 02/03 章 cross-attention
条件注入的地基（本章用不到，读本章前可跳过）

可选：

- [Part 8 06 章 · 推理与量化](../Part8_post_training/tutorial/06_inference_and_serving.md)——
02 章 SD 实操的 fp16 显存策略与它同源（感兴趣再回看）

理论背景

问题引入：为什么需要扩散模型？

生成模型虽然强大，但有两种主流方法：

1. GAN：训练不稳定，模式崩塌
2. VAE：生成质量不高，模糊

扩散模型通过逐步去噪来弥补：

GAN: "直接生成，训练不稳定"
VAE: "压缩再解压，质量不高"
扩散: "逐步去噪，质量高且稳定"

> 类比：GAN 像是直接画画，VAE 像是先拍照再画画，扩散像是先涂满颜料再慢慢擦出画。

数学推导：前向扩散的闭式解

问题设定：

- 原始数据： x_0
- 噪声级别： $t \in \{0, 1, \dots, T\}$
- 噪声 schedule： $\beta_1, \beta_2, \dots, \beta_T$

推导过程：

Step 1: 单步扩散

$$x_t = \sqrt{(1 - \beta_t)} x_{t-1} + \sqrt{\beta_t} \epsilon_t$$

其中 $\epsilon_t \sim N(0, I)$

Step 2: 递推展开

$$\begin{aligned} x_t &= \sqrt{(1 - \beta_t)} x_{t-1} + \sqrt{\beta_t} \epsilon_t \\ &= \sqrt{(1 - \beta_t)} \sqrt{(1 - \beta_{t-1})} x_{t-2} + \dots \\ &= \sqrt{(\bar{\alpha}_t)} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon \end{aligned}$$

$$\text{其中 } \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$$

Step 3: 闭式解

$$x_t = \sqrt{(\bar{\alpha}_t)} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$$

其中 $\epsilon \sim N(0, I)$

关键洞察：

- $\bar{\alpha}_t$ 是"信号保留比例"：t 小信号多， $t \rightarrow T$ 信号趋零
- 闭式解让训练时一步到位，不必迭代 t 次
- 这是扩散模型能高效训练的全部秘密

代码实现

1. 前向：一条"固定的"噪声马尔可夫链

运行 [scripts/01_ddpm_from_scratch.py](../scripts/01_ddpm_from_scratch.py) 验证以下代码。

DDPM (2006.11239) 定义前向过程 q ：逐步给数据加高斯噪声， β_t 是每步的噪声量：

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{(1 - \beta_t)} \cdot x_{t-1}, \beta_t \cdot I)$$

关键推导：代入展开后，任意时刻 t 的边缘分布有闭式解：

$$q(x_t | x_0) = N(x_t; \sqrt{\bar{\alpha}_t} \cdot x_0, (1 - \bar{\alpha}_t) \cdot I) \# \bar{\alpha}_t = \prod \alpha_s \ (\alpha = 1 - \beta)$$

形状追踪：扩散过程

```
| DDPM 扩散过程 |
| |
| 前向扩散（训练时）： |
| x_0: (B, 2) # 原始数据 |
| ↓ 加噪 |
| x_t = √(ᾱ_t) x_0 + √(1 - ᾱ_t) ε |
| x_t: (B, 2) # 噪声数据 |
| |
| 反向去噪（采样时）： |
| x_T: (B, 2) # 纯噪声 |
| ↓ 去噪 |
| x_{t-1} = 1/√α_t · (x_t - β_t/√(1 - ᾱ_t) · ε̂) + √β_t · z # t=0 时 z=0 |
| x_0: (B, 2) # 生成数据 |
| |
| 训练目标： |
| ε_θ(x_t, t) ≈ ε # 预测加进去的噪声 |
```

2. 训练：预测噪声 ε （VLB 的简化形式）

DDPM 论文 § 3.2 把变分下界简化成一个 MSE：让网络从 (x_t, t) 预测加进去的噪声 ε ：

```
python
t = torch.randint(0, T, (B,)) # 随机采 t（闭式让这步 O(1)）
noise = torch.randn_like(x0)
x_t = q_sample(x0, t, noise) # √ᾱ_t · x0 + √(1 - ᾱ_t) · noise
eps_pred = model(x_t, t)
loss = F.mse_loss(eps_pred, noise) # 训练目标：还原"加进去的噪声"
```

- > 签名说明：脚本 01 与本节的 `q_sample(x0, t, noise)` 为省参用模块级
- > `alphas_cumprod`；章末练习与 Assignment 16 改为显式传参的四参版
- > `q_sample(x0, alphas_cumprod, t, noise)`——二者数学完全相同。

实测（2D 双月环玩具，3000 步；RTX 4090, torch 2.6.0+cu124）：denoising loss $1.11 \rightarrow 0.21$ 后稳定（中途在 0.19-0.23 间小幅波动，属随机采 t 的正常现象）。

3. 采样：反向链（论文式 11）

```
x_T ~ N(0, I)
for t in T - 1 ... 0:
     $\hat{\epsilon}_t = \text{model}(x_t, t)$ 
     $x_{t-1} = 1/\sqrt{\alpha_t} \cdot (x_t - \beta_t/\sqrt{1-\alpha_t} \cdot \hat{\epsilon}_t) + \sqrt{\beta_t} \cdot z$  # t=0 时 z=0
```

直觉： $\hat{\epsilon}_t$ 给出“这坨噪声里藏着什么内容”的方向，每步减掉一点、加回一点随机性。

实测（RTX 4090, torch 2.6.0+cu124）：采样 2000 点与真实分布对比——均值偏移 $[0.069, -0.011]$ (≈ 0)、方差比 $[1.039, 1.065]$ (≈ 1) ——分布匹配。

工程实践

调试展示：常见错误与修复

错误 1： β schedule 选择不当

症状：

训练不稳定，或生成质量差

原因： β schedule 太激进（末端噪声过猛）

解法：

python

线性 schedule（简单但末端噪声过猛）

```
beta = torch.linspace(0.0001, 0.02, T)
```

cosine schedule（更平滑，推荐）

```
def cosine_schedule(T, s=0.008):
    steps = torch.arange(T + 1)
    f = torch.cos((steps / T + s) / (1 + s) * math.pi / 2) * 2
    alphas_cumprod = f / f[0]
    betas = 1 - alphas_cumprod[1:] / alphas_cumprod[:-1]
    return torch.clamp(betas, 0, 0.999)
```

错误 2：t 未归一化直接进网络

症状：

,
,
,

训练 loss 居高不下（甚至 NaN），采样输出仍是一团噪声

原因：t 是 0~T-1 的整数（本脚本 T=400），量级远大于网络输入 x_t（±1 量级）——原始 t 直接进 MLP 会淹没 x_t 的信号，时间条件基本没学到，网络分不清噪声级别

解法：

python

先转 float 再除以 T，把 t 压到 0~1（脚本 01 的写法）

temb = self.t_mlp(t.view(-1, 1).float() / T)

真实实现用 sinusoidal 位置编码或 AdaLN 注入 t——思想相同：先编码、再进网络

,

错误 3：t=0 时加噪

症状：

,

生成结果有噪声

,

原因：t=0 时不应该加噪

解法：

python

采样时 t=0 不加噪

```
for t in range(T - 1, -1, -1):
    eps_pred = model(x_t, t)
    if t > 0:
        x_t = (x_t - beta[t] / sqrt(1 - alpha_bar[t]) * eps_pred) / sqrt(alpha[t])
        x_t = x_t + sqrt(beta[t]) * torch.randn_like(x_t)
    else:
        x_t = (x_t - beta[t] / sqrt(1 - alpha_bar[t]) * eps_pred) / sqrt(alpha[t])
```

,

性能数据（实测参考）

方法	训练步数	生成质量	训练时间	说明
DDPM (线性)	3000	良好	<1min	2D 玩具
DDPM (cosine)	3000	更好	<1min	2D 玩具
DDPM (真实图像)	100K+	高	数小时	512×512

> 数据来源：DDPM 论文 + 本课开发机实测

常见陷阱

陷阱 1：β schedule 选择不当

症状：训练不稳定，或生成质量差

原因：β schedule 太激进

解法：使用 cosine schedule

陷阱 2：方差口径不一致

症状：训练 loss 不下降

原因：方差用无偏口径

解法：使用有偏口径

陷阱 3：t=0 时加噪

症状：生成结果有噪声

原因：t=0 时不应该加噪

解法：采样时 t=0 不加噪

最佳实践

配置推荐

参数	推荐值	说明
T	1000	扩散步数
β schedule	cosine	更平滑
学习率	1e-4	Adam 优化器
batch_size	64-256	根据显存调整

学完本部分你能...

- 写出前向闭式并解释 $\bar{\alpha}_t$ 的物理含义（信号保留比例）
- 解释训练目标为什么是"预测噪声"而不是"预测 x_0 "（等价但更稳）
- 手写完整的采样循环（含 t=0 不加噪的细节）
- 回答"为什么扩散训练能一步加噪"（固定高斯链的边际闭式解）
- 识别 β schedule 选择、方差口径等常见陷阱

概念检验

<details>
<summary>Q1: 训练时为什么不迭代 t 次逐步加噪，而是直接采 t 用闭式？</summary>
A: 前向过程没有可学习参数（固定的高斯链），任意 t 的边际有闭式解 → 一步到位。

这让每个训练样本每个 step 都能随机覆盖所有噪声级别， $T=1000$ 也零额外成本。反向过程才有可学习参数（ ϵ 网络），那才是需要迭代的部分（采样时）。

</details>

<details>

<summary>Q2: β schedule（线性 vs cosine）影响什么？</summary>

A: 决定 $\bar{\alpha}_t$ 从 1 降到 0 的速度分布。线性 schedule 在高分辨率下末端噪声过猛（信息丢失过快），cosine（Nichol & Dhariwal 2021）让 $\bar{\alpha}$ 更平滑地衰减，对训练更稳。本课用线性是为了教学直观。

</details>

<details>

<summary>Q3: 为什么训练目标是"预测噪声"而不是"预测 x_0 "？</summary>

A: 两者数学等价，但预测噪声更稳定：

- 预测 x_0 ：需要预测完整的数据，方差大
- 预测 ϵ ：只需要预测噪声，方差小
- 实验表明：预测 ϵ 的训练更稳定，生成质量更高

</details>

动手实践

<details>

<summary>练习 1: 实现前向扩散</summary>

任务：实现一个函数，计算前向扩散的闭式解（签名与 Assignment 16 题 1 完全一致）。

验收标准：

- [] 输入： x_0 (B, 2), alphas_cumprod (T,), t (B,) 长整型, noise (B, 2)
- [] 输出： x_t (B, 2)，逐行满足闭式 $x_t[i] = \sqrt{\bar{\alpha}_{t[i]}} \cdot x_0[i] + \sqrt{(1 - \bar{\alpha}_{t[i]})} \cdot \text{noise}[i]$
- [] 使用闭式解一步到位（不迭代加噪）

步骤提示：

```
python
def q_sample(x0, alphas_cumprod, t, noise):
    """
```

Steps:

1. 取 $s = \text{alphas_cumprod}[t]$, reshape 成 (-1, 1) 以便按行广播
2. 信号项 $s.\text{sqrt}()$ x_0 , 噪声项 $(1 - s).\text{sqrt}()$ noise
3. 返回 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \text{noise}$

```
"""
```

TODO: Implement

```
pass
```

```
,
```

</details>

<details>

<summary>练习 2: 实现采样循环</summary>

任务：实现一个函数，从噪声生成数据。

验收标准：

- [] 输入：模型、T、alpha、beta、alpha_bar
- [] 输出：生成的数据 x_0
- [] 正确处理 t=0 时不加噪

步骤提示：

```
`python
def p_sample_loop(model, T, alpha, beta, alpha_bar):
    """
```

Steps:

1. 从纯噪声开始 $x_T \sim N(0, I)$
 2. 从 T-1 到 0 循环
 3. 预测噪声 $\epsilon_{\text{pred}} = \text{model}(x_t, t)$
 4. 计算 x_{t-1}
 5. t>0 时加噪，t=0 时不加噪
 6. 返回 x_0
- ```
"""
```

## TODO: Implement

```
pass
`
```

</details>

<details>

<summary>练习 3: 实现 cosine schedule</summary>

任务：实现 cosine  $\beta$  schedule。

验收标准：

- [] 输入：T（扩散步数）
- [] 输出：beta (T,)
- [] 使用 cosine 函数

步骤提示：

```
`python
def cosine_schedule(T, s=0.008):
 """
```

Steps:

1. 计算  $\text{steps} = \text{torch.arange}(T + 1)$
  2. 计算  $f = \cos((\text{steps} / T + s) / (1 + s) \pi / 2) * 2$
  3. 计算  $\text{alphas\_cumprod} = f / f[0]$
  4. 计算  $\text{betas} = 1 - \text{alphas\_cumprod}[1:] / \text{alphas\_cumprod}[:-1]$
  5. 返回 betas
- ```
"""
```

TODO: Implement

```
pass
`
```

</details>

课后作业

完成本章后，去 Assignment 16 完成练习：

[Assignment 16](../../assignments/assignment_16/)

下一步

会"从噪声生成分布"了。真实文生图 = 把这套数学搬进 VAE 潜空间 + 用 cross-attention 注入文本条件（Latent Diffusion）——以及 img2img 的 strength 数学。

[02 — 文生图与图生图工具链](02_t2i_i2i_pipelines.md)

02_t2i_i2i_pipelines

02 — 文生图与图生图工具链：Latent Diffusion → SD → img2img

> 从 2D 玩具到真实文生图只差两个工程跃迁：① 搬进 VAE 潜空间（ $8\times$ 空间压缩， 512^2 图像 → 64×64 潜变量）、② 用 cross-attention 注入文本条件（CLIP/T5 嵌入做 > K/V，Part 15 对齐空间的直接消费）。工具锚点：diffusers（34.4k，全谱系支持）。

学习目标

完成本章后，你将能够：

- 解释 Latent Diffusion 的两个工程跃迁（潜空间压缩 + cross-attention 条件）及其收益
- 推导 img2img 的 strength → t。映射，并用 $\bar{\alpha}_t$ 解释 strength 两端的行为
- 运行 diffusers 完成 SD1.5 文生图与图生图（正确处理模型 ID、dtype、seed）
- 区分 prompt（语义）/ ControlNet（结构）/ strength（保留度）三条控制通道
- 识别 strength 取值极端、ControlNet 权重过高等陷阱并给出参数修正

前置知识

- 01 章：DDPM 三段数学；Part 15 02 章：文本对齐空间（cross-attention 消费它）

1. Latent Diffusion（2112.10752）：为什么搬进潜空间

像素空间扩散 $512^2 \times 3 = 786\text{K}$ 维/图 → 太贵

VAE 编码到 $64\times 64\times 4 = 16\text{K}$ 维 → $8\times$ 空间压缩（论文：感知无损）

扩散在潜空间进行；生成后 VAE 解码回像素

条件注入：cross-attention——图像潜变量的 Q，文本嵌入的 K/V（Part 16 脚本 02 的 ① 就是它的最小版）。SD1.5 用 CLIP 文本塔；SDXL 双塔；SD3/FLUX 用 T5。

2. 模型谱系与 4090 跑法

机制	4090 24GB	备注
像素空间	CPU	教学数学
LDM + U-Net + CLIP	fp16 ~2GB	△ 用 `stable-diffusion-v1-5/stable-diffusion-v1-5` (run
双文本塔 + 更大 U-Net	fp16 ~7GB	
Rectified Flow + MMDiT (文本/图像分块独立权重、双向 joint attention)	fp8 ≈12GB	FLUX 12B DiT ; dev 非商用/schnell Apache

```
`bash
```

工具实操（独立 venv，与训练环境隔离）

```
pip install diffusers transformers accelerate
```

```
`python
```

vllm 在这里帮不上忙——文生图就是 diffusers 两行：

```
from diffusers import StableDiffusionPipeline
pipe = StableDiffusionPipeline.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5", torch_dtype=torch.float16).to("cuda")
image = pipe("a photo of an astronaut riding a horse", guidance_scale=7.5).images[0]
```

- Rectified Flow (SD3/FLUX)：把"数据 $\leftrightarrow$ 噪声"的路径拉直成直线，训练目标改为预测速度场 $v = \epsilon - x_o$ ，采样路径更直 $\rightarrow$ 步数更少。MMDiT = 文本与图像 patch 各自独立权重 + 双向 joint attention（不再是单向 cross-attention）——条件与生成的对齐从"单向消费"升级为"双向融合"。

3. 图生图：strength 参数的数学（对 01 章闭式的直接复用）

img2img 不是"重画"，而是从参考图的部分噪声起步：

```
t_o = num_inference_steps * strength # strength ∈ (0,1]
x_{t_o} = \sqrt{\alpha_{t_o}} \cdot \text{encode(参考图)} + \sqrt{(1 - \alpha_{t_o})} \cdot \epsilon # \leftarrow 01 章 q_sample 的直接调用！
从 t_o 反向去噪到 0（跳过前段，保留参考图内容结构）
```

- strength=1 $\rightarrow$ 从纯噪声起步（= 文生图）；strength $\rightarrow$ 0 $\rightarrow$ 几乎照抄参考图。
- 这就是"01 章闭式公式的第二次消费"：训练用它高效加噪，img2img 用它构造起点。

4. ControlNet：空间结构控制（旁路注入）

ControlNet (2302.05543, 34.1k)：把冻结的 SD 编码块复制一份作为可训练副本，条件 (canny/深度/姿态/涂鸦) 经副本处理后注入原网络——注入点是零卷积 (1 $\times$ 1、零初始化：训练起点不扰动原模型——LoRA B=0 的同款设计模式！)。

用途：让生成严格服从边缘图/骨架图——"prompt 控制语义，ControlNet 控制结构"。

工程实践

常见陷阱

陷阱 1：strength=1.0 时输出与参考图毫无关系 ("噪声把参考图全破坏")

症状：img2img 结果不含参考图的任何构图/物体，看起来就是一张普通文生图。

原因： $t_0 = steps \times 1.0$ = 全部步数，起点 $x_{\{t_0\}} = \sqrt{\alpha_{\{t_0\}}} \cdot encode(\text{参考图}) + \sqrt{(1 - \alpha_{\{t_0\}})} \cdot \varepsilon$ 中的 $\alpha_{\{t_0\}} \rightarrow 0$ (§ 3 的公式) ——参考图信号几乎被噪声淹没。

解法："保留构图、改变内容"用 0.4-0.7；要彻底重画不如直接用文生图管线（还省一次参考图的 VAE 编码）。动手实践练习 1 的 $\sqrt{\alpha}$ 表可以提前算出每个 strength 保留多少信号。

陷阱 2：加载 SD1.5 报 401/404 (repo not found)

症状：

OSError: runwayml/stable-diffusion-v1-5 is not a local folder and is not a valid huggingface.co repository

原因：runwayml 原模型 ID 已因版权问题从 Hub 删除（社区迁移到镜像 ID）。

解法：换社区镜像 [stable-diffusion-v1-5/stable-diffusion-v1-5](#) (§ 2 表格已标注)。

陷阱 3：ControlNet 权重过高——输出贴死条件图、prompt 失效

症状：生成严格"描"出 canny 边缘，边缘处发黑、纹理过曝（"烧焦"），改 prompt 几乎无反应。

原因：[controlnet_conditioning_scale](#) 过大 → 旁路残差压过主干输出，模型被结构条件"锁死"（零卷积只保证训练起点不扰动，不限制推理时手动调高权重）。

解法：0.5-0.8 起步逐步上调；结构约束过强时先降 ControlNet 权重，而不是加大 guidance。记住分工："prompt 控制语义，ControlNet 控制结构" (§ 4)。

最佳实践：SD 推理配置推荐 (24GB 单卡起步值)

参数	推荐值	说明
dtype	fp16	SD1.5 ~2GB / SDXL ~7GB，无感质量损失
采样步数	25-30	配 DPM-Solver++ / DDIM，再加步收益递减
guidance_scale	7-8	SD 默认 7.5；w 过大的代价见 03 章 CFG
img2img strength	0.4-0.7	"保留构图、改变内容"的工程常用档（陷阱 1）
ControlNet scale	0.5-0.8 起步	过高被条件"锁死"（陷阱 3）
对比实验	固定 generator seed	一切可复现对比的前提

学完本部分你能...

- 说清 Latent Diffusion 的两个跃迁（潜空间 + cross-attention 条件）
- 在 4090 上跑通 SD1.5 文生图（含正确模型 ID）
- 推导 img2img 的 $\text{strength} \rightarrow t_0$ 映射，解释 strength 的两端行为
- 说出 ControlNet 零卷积的设计意图（与 LoRA B=0 归入同一模式）

概念检验

<details>

<summary>Q1: img2img 的 strength 太小（如 0.1）会发生什么？太大呢？</summary>

A: $t_0 = \text{steps} \times 0.1 \rightarrow$ 几乎从干净潜变量起步，只去掉最后几步噪声——输出 $\approx$ 参考图（变化微小）； $\text{strength} \rightarrow 1$ 则结构信息全丢（=纯文生图）。工程上 0.4-0.7 是“保留构图、改变内容”的常用档位。

</details>

<details>

<summary>Q2: Rectified Flow 相比 DDPM 的“直路”优势在采样上怎么体现？</summary>

A: DDPM 的概率流轨迹弯曲，需要几十步数值积分；RF 的直线路径让 ODE 求解步数大幅减少（SD3/FLUX 常用 <30 步甚至蒸馏到 4 步），且理论分析更简单。

</details>

<details>

<summary>Q3: 为什么在 VAE 潜空间而不是像素空间做扩散？</summary>

A: 像素空间 $512^2 \times 3 = 786\text{K}$ 维/图，U-Net 每一步的计算与显存成本爆炸；VAE 编码到 $64 \times 64 \times 4 = 16\text{K}$ 维（ $8 \times$ 空间压缩，论文实测感知近无损）后，训练/采样成本低一个量级（§1 的对比）。代价：细节上限受 VAE 重建误差约束——手部、文字这类高频细节的失真多来自 VAE 而非扩散过程本身。

</details>

<details>

<summary>Q4: ControlNet 的零卷积为什么初始化为 0？</summary>

A: 旁路输出从“恒为 0（完全不扰动原模型）”的状态起步微调，避免随机初始化的副本一开始就破坏预训练能力——与 LoRA 的 B=0 是同一设计模式：**新增模块从零映射起步，把保护基座的责任放进初始化里**（§4）。

</details>

动手实践

练习 1： $\text{strength} \rightarrow t_0 \rightarrow$ 信号保留比例表（CPU 纯数学，无需 GPU）

任务：用 01 章的线性 β schedule 和 $\bar{\alpha}$ 闭式，把 §3 的 $\text{strength} \rightarrow t_0$ 映射量化成“信号保留表”——把陷阱 1 的“噪声全破坏”变成数字。

验收标准：

- [] $T=400$ 、 $\text{num_inference_steps}=30$ ，输出 $\text{strength} \in \{0.2, 0.5, 0.8, 1.0\}$ 四行：
 $\text{strength} / t_0 / \sqrt{\bar{\alpha}_{\{t_0\}}}$
- [] $\sqrt{\bar{\alpha}}$ 随 strength 单调递减； $\text{strength}=1.0$ 时 ≈ 0.13 （信号只剩 ~13%）
- [] 用一句话解释：为什么 $\text{strength}=1.0$ 等价于纯文生图

步骤提示：

`python

```
import torch
betas = torch.linspace(1e-4, 0.02, 400) # 01 章的线性 schedule
alpha_bar = torch.cumprod(1 - betas, dim=0)
for s in [0.2, 0.5, 0.8, 1.0]:
    t0 = int(30 * s) # § 3 : t_0 = steps × strength
    idx = min(int(t0 / 30 * 399), 399) # 推理步 → 400 步扩散时间线
    print(f"strength={s:.1f} t0={t0:2d} sqrt_abar={alpha_bar[idx].sqrt():.4f}")
```

> 参考数值（本课开发机 CPU 实算）：0.9204 / 0.6017 / 0.2737 / 0.1322。

练习 2（操作型，需 GPU + 独立 venv）：SD1.5 img2img strength 扫参

任务：同一参考图 + 同一 prompt + 同一 seed，strength $\in \{0.2, 0.5, 0.8, 1.0\}$
各生成一张，做"结构保留 vs 语义改变"的定性记录表。

验收标准：

- [] 产出 4 张图 + 一张记录表：strength / t_0 = 30 · s / 构图保留度 / prompt 遵循度
- [] 固定 seed (`torch.Generator("cuda").manual_seed(42)`)，重跑得到相同 4 张图
- [] 记录表的趋势与练习 1 的 $\sqrt{\alpha}$ 数值一致：strength 越大，构图保留越低
- [] 模型 ID 使用镜像 [stable-diffusion-v1-5/stable-diffusion-v1-5](#)（陷阱 2）

步骤提示：

```
python
from diffusers import StableDiffusionImg2ImgPipeline
import torch
pipe = StableDiffusionImg2ImgPipeline.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5", torch_dtype=torch.float16).to("cuda")
for s in [0.2, 0.5, 0.8, 1.0]:
    g = torch.Generator("cuda").manual_seed(42)
    img = pipe("a fantasy landscape, cinematic lighting", image=init_image,
        strength=s, num_inference_steps=30, generator=g).images[0]
    img.save(f"out_strength_{s}.png") # 逐张回填记录表
```

进阶与缺口（面试向：本课未深挖的高频考点）

- 采样器：DDIM = 把 DDPM 的随机反向改成确定性 ODE 步进（可跳步：从 t 直接跳到 t - 20，eta=0 时同一 x_T 结果可复现）；DPM-Solver 系 = 高阶 ODE solver，10-15 步达到 DDPM 25 步质量。面试一句话："DDIM 解决确定性/跳步，DPM-Solver 解决步数效率"。
- 生成评估：FID（真实/生成样本在 Inception 特征空间的高斯距离，越低越好，但对模式坍塌不敏感）；CLIP-score（图文一致性）；HPS/人评（美学）。生成侧没有单一指标——组报数 + 人评抽样是行业实践（呼应 Part 8 07 章）。
- 扩散模型微调：SD LoRA（与 Part 8 08 章同机制，target 注入 U-Net 的 attention 层）、Textual Inversion（学一个新 token 的 embedding）、DreamBooth（少量图全参微调 + class-specific prior preservation）——LLaMA-Factory/diffusers 均可训练，4090 可跑。
- 蒸馏与步数压缩：LCM/Turbo/DMD 把几十步蒸馏到 1-4 步——生产部署的标配方向（呼应 Part 14 的吞吐优化：生成侧同样有"步数 vs 质量"的 goodput 权衡）。
- inpainting / 外扩 / 潜空间插值 (slerp)：工程日常三件套，diffusers 均有现成 pipeline，机制都在 01 章 q_sample 的框架内。

课后作业

[Assignment 16](../../assignments/assignment_16/)

下一步

文生图会了，参考图怎么"指挥"生成（多图参考/人物一致性）？以及视频怎么在图像模型上加"时间维度"？——这些全是特征对齐的不同形态。

[03 — 特征对齐与视频生成](03_alignment_and_video.md)

03_alignment_and_video

03 — 特征对齐与视频生成：从 IP-Adapter 到 Wan2.1

- > 收官章。三件事：① 手写解耦交叉注意力（IP-Adapter 的核心）——参考图特征
- > 注入的教科书案例（跑 [scripts/02_alignment_mechanisms.py](../scripts/02_alignment_mechanisms.py)）；
- > ② CFG 的外推数学；③ 视频生成——图像模型加"时间维度"的最小增量。

学习目标

完成本章后，你将能够：

- 手写解耦交叉注意力（IP-Adapter 核心），解释"仅 22M 参数、基座冻结"何以可能
- 写出 CFG 外推公式，解释 w 的权衡与训练侧配套（~10% 条件置空）
- 应用"图像模型 + temporal attention"的最小增量视角拆解视频生成管线
- 选型 24GB 单卡上的视频模型（CogVideoX-2B / Wan2.1-1.3B / HunyuanVideo 量化）
- 识别 CFG 过强、IP-Adapter scale 过大、视频帧间闪烁等陷阱并给出修正

前置知识

- 02 章：cross-attention 条件注入；Part 15 02 章：对齐损失（本章的"生成侧"呼应）

1. 解耦交叉注意力：参考图作为"类文本 token"

问题：想让生成严格遵循一张参考图（人物/风格/物体），微调整个模型太贵且会遗忘；直接把参考 token 拼进文本序列会干扰原模型的文本能力。

IP-Adapter (2308.06721, 仅 22M 参数) 的解法：参考图经 CLIP 图像编码器提特征，为一套全新的独立 K/V 投影（原模型权重冻结不动）：

```
out = attn(Q, K_txt, V_txt) + scale * attn(Q, K_ref, V_ref)
↑ 独立新增的投影（参考图专用）
```


脚本 02 的实测：scale=0 时输出与纯文本条件完全一致（原行为不变）、scale 增大参考影响线性增强——"解耦"= 保留基座能力 + 强度可调 + 可与 ControlNet 正交组合。

- 这就是跨模态特征对齐的生成侧形态：把参考图的嵌入投影进文本条件所在的 token 空间 ("类文本 token")，让扩散网络的 cross-attention 像消费文本一样消费它。
- 变体谱系：IP-Adapter Plus（细粒度）、FaceID（ArcFace 人脸嵌入）、InstantID（IdentityNet+人脸嵌入，单照片免调）、PuLID（对比对齐，保护可编辑性，有 FLUX 版）。

2. CFG：条件引导的外推数学

$$\epsilon = \epsilon_{\text{uncond}} + w \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}}) \# w = \text{guidance scale (SD 默认 7.5)}$$

- $(\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$ 是"条件方向"——w 放大这个方向的步长。
- 脚本 02 实测：w=7.5 时输出与条件方向的余弦 ≈ 0.998 （外推方向正确）。
- $\triangle$ w 过大 $\rightarrow$ 过饱和/失真（外推出训练分布）；这就是 Part 8 07 章"goodput 思维"的生成版：不是越引导越好，是指令遵循与自然度的权衡。
- 训练侧配套：训练时以 $\sim 10\%$ 概率把条件置空（uncond） $\rightarrow$ 让模型两种模式都会——这是"分类器引导"进化为"无分类器引导"的关键。

3. 视频生成：图像模型 + 时间维度

组件	图像模型（SD 系）	视频模型（Latte/CogVideoX/Wan）
压缩	2D VAE（空间）	**3D Causal VAE**（空间+时间一起压）
去噪骨干	2D U-Net / DiT	同款 + **temporal attention**（空间块间插入时间轴注意力）
条件	文本 cross-attention	文本 + 可选首帧/尾帧（图生视频）

- 最小增量视角：视频 = 把图像的 (B, T_frame, C, H, W) 潜变量 reshaping 成 (B $\times$ T_frame, C, H, W) 做空间注意力，再 reshape 回 (B, T_frame, C $\times$ H $\times$ W) 做**时间轴注意力**——空间块之间插入一层"帧间交流"。CogVideoX 的 expert adaptive LayerNorm、Wan2.1 的 flow matching + 文本编码器升级（UMT5），都是在此骨架上的强化。
- 24GB 实测路径（都有官方/社区 diffusers 支持）：
- CogVideoX-2B（Apache-2.0）：fp16 $\sim$ 4GB、int8 3.6GB——文生视频/图生视频的教学首选，连 1080Ti 都能跑
- Wan2.1-1.3B（Apache-2.0）：8.2GB，4090 上 $\sim$ 4 分钟出 5 秒 480p——质量最强的 24GB 选项
- HunyuanVideo（13B）：720p 需 $\sim$ 60GB，社区量化可到 24GB——引述不实操

4. 跨模态对齐主线（Part 15+16 收官总图）

理解侧（Part 15） 生成侧（Part 16）
图像 $\rightarrow$ ViT $\rightarrow$ projector $\longrightarrow$ 文本 $\rightarrow$ CLIP/T5 $\rightarrow$ K/V $\longrightarrow$
▼▼
LLM token 空间 扩散条件空间



参考图 → CLIP → IP-Adapter KV ————— (本脚本②)

对齐三件套：翻译器 (projector/adaptor KV) + 对齐训练 (Stage1/adaptor 训练)
+ 可控强度 (scale/CFG) ——理解与生成共享同一套设计模式

工程实践

常见陷阱

陷阱1：CFG的w过大——过饱和/失真

症状：颜色过饱和、对比度过高、细节出现"塑料感"伪影； $w \geq 15$ 时肉眼可见失真。

原因： ϵ 沿 (cond - uncond) 方向外推，w 越大离训练分布越远 (§ 2 的 $\triangle$) ——方向对了，但步长过头。

解法：回落到 6-8 (视频 5-7)。想要更强的指令遵循，优先改 prompt / negative prompt，而不是拉满 w。

陷阱2：IP-Adapter的scale过大——"抄死"参考图

症状：输出几乎复刻参考图，文本指令失效，还可能出现分布外伪影。

原因：§ 1 的解耦公式是线性叠加 $\text{attn_txt} + \text{scale} \cdot \text{attn_ref}$ ，scale 过大时参考分支注意力压过文本分支。

解法：0.5-1.5 起步按效果调 (与 CFG 的 w 同理：引导强度与可控性的权衡)。

陷阱3：视频 temporal 一致性差——帧间闪烁/物体跳变

症状：帧间物体身份、颜色跳变，背景周期性闪烁，动作不连贯。

原因：帧与帧之间缺少信息交流——逐帧独立解码的 VAE、没有 temporal attention 的图像模型直连视频，或去噪步数不足导致高频时序噪声残留。

解法：用带 3D causal VAE + temporal attention 的模型 (CogVideoX / Wan2.1, § 3 的管线)；提高去噪步数 (50 起步)、必要时降帧数/分辨率；可控性优先的场景用图生视频锚定首帧。

最佳实践：对齐机制参数推荐 (起点值)

机制	推荐起点	过头的症状
CFG w	图像 7-8 / 视频 5-7	过饱和 (陷阱 1)
IP-Adapter scale	0.5-1.5	抄死参考图 (陷阱 2)
ControlNet scale	0.5-0.8 (02 章陷阱 3)	被条件图"锁死"
视频去噪步数	50 (质量) / 30 (快)	步数不足 → 闪烁 (陷阱 3)
24GB 视频选型	CogVideoX-2B (教学) / Wan2.1-1.3B (质量)	HunyuanVideo 需量化

- > 三条对齐通道（CFG / IP-Adapter / ControlNet）彼此正交，可以组合使用——
- > 逐个从起点值调起，一次只动一个参数。

学完本部分你能...

- 手写解耦交叉注意力，说清 IP-Adapter"22M 参数不动基座"的原理
- 写出 CFG 公式并解释 w 的权衡与训练侧配套（条件置空）
- 用"图像模型 + temporal attention"的最小增量视角理解视频生成
- 在 24GB 上选型：CogVideoX-2B / Wan2.1-1.3B / HunyuanVideo 量化

概念检验

<details>

<summary>Q1: IP-Adapter 的 scale 设很大（如 10）会怎样？为什么？</summary>

A: 参考分支的注意力权重压过文本分支——生成"抄死"参考图、文本指令失效，且可能跑出分布外伪影。与 CFG 的 w 过大同理：引导强度与自然度是权衡，实践上 0.5-1.5 起步按效果调。

</details>

<details>

<summary>Q2: 视频模型的 temporal attention 为什么通常"跳过第一帧"或用因果化设计？</summary>

A: 与文本因果遮罩同源：自回归/可控生成的场景下，未来帧不应影响已确定的帧；另外非因果的全帧注意力训练成本高（帧数平方）。CogVideoX 用 3D 因果 VAE + 分层策略平衡质量与成本。

</details>

<details>

<summary>Q3: 训练扩散模型时为什么以 ~10% 概率把条件置空？不做会怎样？</summary>

A: CFG 采样要同时算 ϵ_{cond} 和 ϵ_{uncond} 做外推（§2 公式），模型必须"两种模式都会"。不做置空，模型从未见过空条件 $\rightarrow$ uncond 分支输出失真，外推方向 $(\text{cond} - \text{uncond})$ 被污染，引导越强伪影越大。这 10% 是 CFG 的训练侧配套，不是普通的数据增强。

</details>

动手实践

练习 1：CFG 外推方向的数值验证（CPU 纯数学，无需 GPU）

任务：复现脚本 02 的 [3] 号实验——随机两路 ϵ ，扫 $w \in \{1, 3, 7.5, 15\}$ ，计算外推结果与条件方向 $(\text{cond} - \text{uncond})$ 的余弦，找到"方向稳定"的 w 区间，并对照陷阱 1 理解"大 $\neq$ 更好"。

验收标准：

- [] 输出 w / 余弦 两列的表，4 行
- [] $w=1$ 时余弦明显偏低（随机两路下 ≈ 0.7 ）， $w \geq 7.5$ 时 > 0.99 （外推方向收敛）
- [] 一句话回答： w 越大越贴条件方向，为什么实践中不把 w 拉满？

步骤提示：

```
python
import torch, torch.nn.functional as F
torch.manual_seed(1337)
u, c = torch.randn(1, 512), torch.randn(1, 512)
d = c - u # 条件方向
for w in [1, 3, 7.5, 15]:
    eps = u + w * d # CFG 外推 ( § 2 公式)
    cos = F.cosine_similarity(eps, d, dim=-1)
    print(f"w={w:5.1f} cos={cos.item():.4f}")
```

> 参考数值 (seed=1337, 本课开发机 CPU 实算) : 0.7004 / 0.9803 / 0.9974 / 0.9994。

练习 2 (操作型, 需 GPU) : 视频生成最小闭环 + 一致性观察

任务: 用 CogVideoX-2B (教学首选) 生成两段 ~5 秒视频: 默认参数一段; 只改一个参数 (去噪步数 50→25 或 guidance 6→9) 再一段, 对照观察 temporal 一致性差异。

验收标准:

- [] 产出 2 段视频 + 记录表: 参数 / 生成时长 / 帧间一致性 (物体身份、背景闪烁) / 首帧是否漂移
- [] 能指出管线的三个组件位置: 3D causal VAE (压缩)、temporal attention (帧间交流)、文本 cross-attention (条件) —— 对照 § 3 表格
- [] 记录"参数改动 → 一致性变化"的对应关系 (如步数减半 → 闪烁增多, 对应陷阱 3)

步骤提示:

```
python
from diffusers import CogVideoXPipeline
import torch
pipe = CogVideoXPipeline.from_pretrained(
    "THUDM/CogVideoX-2b", torch_dtype=torch.float16).to("cuda")
video = pipe("a panda dancing in a bamboo forest", num_frames=49,
    num_inference_steps=50, guidance_scale=6.0).frames[0]
```

导出: `from diffusers.utils import export_to_video;`
`export_to_video(video, "out.mp4")`

第二段只改

`num_inference_steps=25`, 其余不动 (一次只动一个变量)

,

课后作业

[Assignment 16](../assignments/assignment_16/)

生成线毕业 (Part 1-16) —— 但故事没完

Part 1-16: 从手写 bigram 到多模态与生成——理解侧 (15)、生成侧 (16) 双线收拢。
 到这一步, 语言、理解、生成三块基石都已就位。**生成模型已经能"画", 下一步让它学会

"动手"——调用工具、多轮决策**：Part 17 用 RL（GRPO 的 agent 版）训练模型自己拆解任务、调用工具、从环境反馈中改进。

[Part 17 — Agentic RL：从单轮对话到会调工具的智能体](../Part17_agentic_rl/tutorial/README.md)

- > 面试备战 ([docs/llm_interview_guide.md](../docs/llm_interview_guide.md)) 、
- > 论文训练 ([docs/paper_reading_guide.md](../docs/paper_reading_guide.md))
- > 随时可取；想继续深挖工程侧，GPUMODE / Ultra-Scale Playbook 是下一层。

[← 上一章：文生图与图生图](02_t2i_i2i_pipelines.md) | [Part 16 README](README.md) | [下一站：Part 17 Agentic RL →](../Part17_agentic_rl/tutorial/README.md)